Technische Universität München

TUM School of Computation, Information and Technology

CPS Project Technical Report

Autonomous Plant Keeper

Course: Embedded Systems, Cyber-Physical Systems and Robotics

Members:

Mia Sara Christina Dreier

Beyrem Hadj Fredj

Jen-Chien Hou

Helen Nike Kensbock

Tzuhung Lin

Hussein Nagi

Thanh Tuan Kiet Nguyen

September 8, 2026

Table of Contents

1 Introduction

Plants depend on environmental conditions such as light, temperature, humidity, and soil moisture to maintain healthy growth. These conditions can change over time, and some more sensitive species may require continuous intervention. Manual monitoring can provide only intermittent information, requires the user to respond to changing conditions, and has little tolerance for the user being absent for a long period. This motivates the development of an automated system capable of continuously observing the plant environment and assisting in maintaining suitable conditions.

This project presents a cyber-physical plant health monitoring system that combines environmental sensing, embedded computation, data visualization, and physical actuation. The system uses sensors to observe relevant environmental conditions around a plant and an embedded controller to process these measurements. The collected information is made available through a user interface, while actuators can influence the physical environment of the plant. In this way, the system connects the physical plant and its environment with a corresponding cyber system and enables interaction between the two.

The objective of the project is therefore to develop a functional prototype that can monitor the plant's environmental conditions, provide the collected information to the user, and autonomously respond to selected conditions through physical actuators. This objective is inspired from an actual industrial practice called predictive maintenance, where actual machines are abstracted into a "digital twin" on a digital device, and then monitored and controlled by embedded systems. In our case, the core idea is very similar, but instead of monitoring a complicated machine, we monitor a plant.

The remainder of this report describes the requirements and design of the system, its hardware and software architecture, and the testing and evaluation of the resulting prototype.

2 System Requirements and Design Goals

The system is designed as an autonomous plant monitoring and care system that continuously observes the environmental conditions of a plant and provides automated responses when required.

The primary functional requirement is continuous monitoring of the environmental variables relevant to plant growth. The system therefore measures soil moisture, ambient temperature, relative humidity, air pressure, and light intensity using dedicated sensors.

A second requirement is the automatic control of the plant environment. Based on the measured conditions, the system should be able to autonomously control physical

actuators, in particular the supplemental grow light and the irrigation system. This establishes a feedback loop in which measurements of the physical environment can result in actions that modify that environment.

The system also requires communication and remote monitoring. Sensor measurements and relevant system states are to be transmitted from the embedded controller to a cloud-based backend, and a web-based frontend provides a user-facing visualization and summary of the collected data and actuator states.

In addition to these functional requirements, the physical implementation must allow the components to operate together reliably in a plant environment. Sensors need to be positioned such that they can obtain meaningful measurements, while the electronic components must be mechanically secured and enclosed.

Based on these requirements, the resulting prototype should demonstrate a complete interaction loop between the physical plant and its cyber representation: environmental conditions are measured by sensors, processed by the embedded controller, communicated to the software system, presented to the user, and used to control physical actuators.

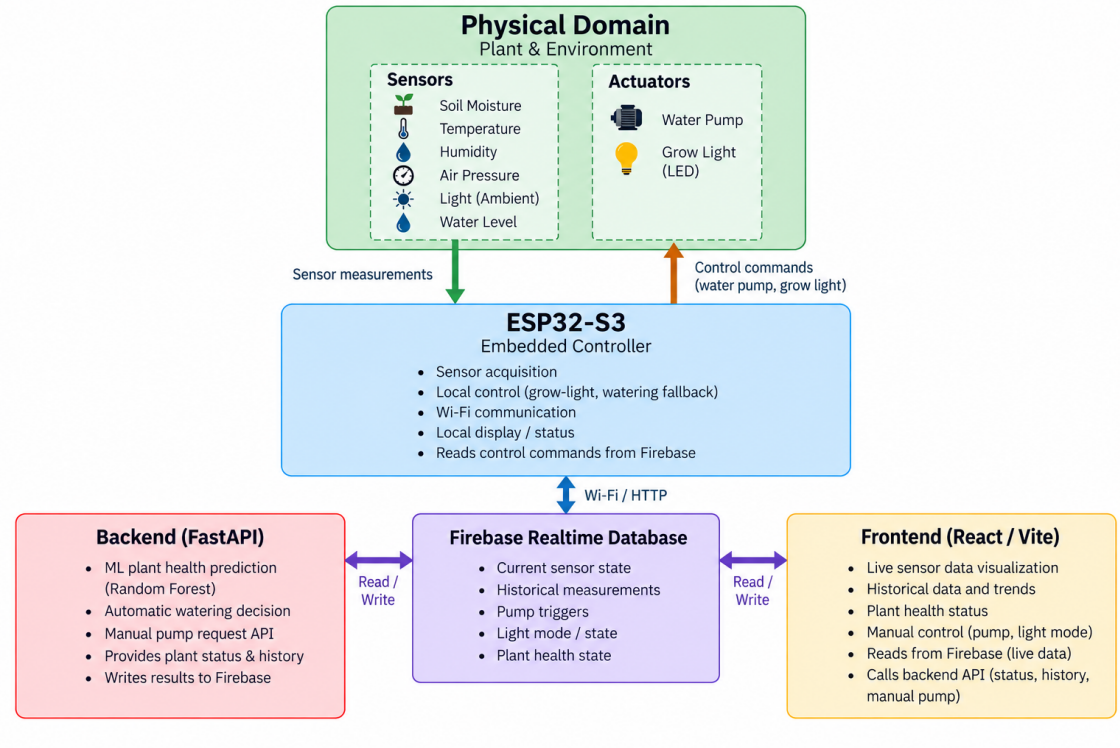
3 System Architecture

The Autonomous Plant Keeper is implemented as a distributed cyber-physical system that connects a physical plant environment with embedded computation, networked data processing, and a user-facing software system. The architecture consists of four main functional layers: the physical plant and its sensors and actuators, the ESP32-S3 embedded controller, the cloud-based data layer and backend, and the web-based frontend. These components interact continuously to monitor the plant, determine its current condition, and influence its physical environment.

At the physical level, the system observes the plant through several sensors. The ESP32-S3 acquires soil moisture, temperature, humidity, air pressure, and light measurements, as well as the state of the water reservoir. The measured data forms the system's observation of the physical environment. The controller also interfaces with the two main actuators, the water pump and grow light.

The ESP32-S3 forms the embedded core of the system. It periodically acquires sensor measurements and publishes them to Firebase. It also executes local control logic such as grow-light operation and watering mechanisms. Thus, the embedded system is not merely a data-collection device but participates directly in the control of the physical plant.

Overall Architecture of the Autonomous Plant Keeper



Firebase acts as the shared data and communication layer between the distributed components. Current sensor measurements and historical records are published by the ESP32, while control information and derived plant state are stored in corresponding database entries. The database therefore provides a common representation of the system state. The communication is consequently asynchronous: the different components periodically read or react to information available in the shared data layer rather than executing a single synchronized reaction.

The FastAPI backend provides the main centralized processing functionality. It retrieves the latest sensor measurements from Firebase and classify its health status. The resulting state is published back to Firebase and can subsequently be displayed. The backend also performs the primary automatic watering decision based on the soil-moisture threshold. This trigger is subsequently consumed by the ESP32, which performs the physical actuation.

The React frontend provides the user-facing interface to the cyber system. It obtains live sensor data and actuator state from Firebase provides visualization of measurements and derived information. User can interact with the system through the frontend features, such as controlling lighting and watering.

The resulting architecture forms a distributed feedback loop between the physical and cyber domains:

physical environment → sensing → embedded processing → shared cyber state → analysis/decision → control command → embedded controller → actuation → physical environment.

This structure reflects the CPS view of systems as interacting computational and physical components. In particular, the system can be understood in terms of reactive behavior and state evolution: sensor observations cause the embedded system and backend to update their respective states, while control decisions result in changes to the physical environment.

The architecture directly addresses the requirements defined in Section 2. The sensor subsystem and periodic embedded measurements provide continuous environmental monitoring. The backend's and ESP32's automatic watering and lighting logic provide autonomous plant care. Firebase, the backend, and the frontend provide the complete communication, data storage, remote monitoring, and visualization pipeline. The combination of these features establishes the required interaction and feedback loop between the cyber system and the physical plant rather than treating monitoring and control as independent functions.

4 Hardware Design and Physical Integration

Autonomous Plant Keeper Full View

The hardware of the Autonomous Plant Keeper is designed around the ESP32-S3 DevKitC-1 as the central embedded controller. The physical system combines environmental sensing, actuation, local status feedback, and supporting mechanical structures into a single, straightforward plant-care setup.

4.1 Embedded Controller and Sensor Subsystem

The ESP32-S3 DevKitC-1 was selected as the central controller because it provides the required GPIO, ADC, I^2C, and Wi-Fi interfaces in a single compact development board. It therefore serves as the interface between the physical hardware and the higher-level software system without requiring an additional communication module.

Four environmental quantities are measured directly. A BME280 provides temperature, relative humidity, and air pressure measurements over the I^2C bus, while a BH1750 measures ambient light intensity over the same bus. Both sensors share the ESP32-S3's I^2C lines, using GPIO 8 for SDA and GPIO 9 for SCL. Soil moisture is measured using an AZDelivery capacitive soil-moisture sensor connected to an analog input on GPIO 1. The raw ADC measurement is converted into a relative soil-moisture percentage using experimentally defined dry and wet calibration values. In addition, a digital water-level switch connected to GPIO 5 detects whether the water reservoir is empty. This information is used as a safety condition for the pump.

The BME280's air-pressure measurement is recorded together with the other sensor values, although it is not currently used as a direct input to the plant-health classification or actuator-control logic. Similarly, the light sensor measures two quantities: *effective illuminance*, whose measurements comprise both room light and the lamp's contribution, and *ambient illuminance*, which is simply the room light, and is measured with the grow light deliberately switched off at the moment of sensing. The latter measurement is used to determine whether the grow light should operate.

4.2 Actuation and Electrical Integration

The system has two primary actuators: a water pump and a grow light. Since these loads cannot be driven directly by the ESP32-S3's GPIO pins, each is controlled through a dedicated relay. The two relay control signals are connected to GPIO 2 and GPIO 4, respectively. The pump and relay modules use an external 12 V supply, while the ESP32-S3 and low-power sensors are supplied separately. This separates the low-voltage control electronics from the higher-power actuator circuit.

The water pump draws water from the reservoir through a flexible tube positioned toward the plant. The water-level switch provides an additional hardware-level constraint: if the reservoir is detected as empty, the controller prevents the pump from being activated.

The grow light is similarly switched through a relay. Its physical position is determined by the plant frame described below, allowing the light source to remain above and around the plant at a fixed position.

In addition to the sensors and actuators, the embedded assembly contains a local display and status indicator. The current firmware uses an ILI9341-based TFT display to show sensor values, system status, and actuator-related states. A single RGB NeoPixel provides additional status feedback, particularly for the network and sensing state. These components allow important system information to be observed directly at the physical device without relying exclusively on the web interface.

4.3 Mechanical Structure

The plant itself is supported by a simple, self-designed wooden frame. The frame provides the mechanical structure required to position the components around the plant while keeping the overall construction lightweight and easy to assemble. In particular, the upper part of the frame provides a mounting point from which the grow light can be suspended above the plant.

The physical arrangement also separates the plant and water-handling components from the main electronics. This is particularly important because the system combines a water reservoir and pump with electrical components. Keeping the electronic enclosure away from the direct water path reduces the risk of accidental water exposure while still allowing the sensors and actuators to be connected through routed cables and tubing.

4.4 Electronics Enclosure and System Integration

The ESP32-S3 and its associated electronic components are housed in a custom 3D-printed case resembling BMO, a character from Adventure Time. It provides a compact physical housing for the controller and supporting electronics while exposing the required interfaces for sensor connections, actuator wiring, power, and the local display.

The enclosure is an important part of the integration between the electrical and mechanical subsystems. Instead of leaving the ESP32, relay-control wiring, and associated connections exposed on a breadboard, the final assembly places the main electronics into a dedicated enclosure while the sensors and actuators remain positioned where they are physically useful. This results in a separation between the electronics subsystem, which is concentrated around the ESP32-S3 and its enclosure, and the plant-facing subsystem, which consists of the sensors, pump, tubing, and grow light.

The resulting hardware can therefore be viewed as three closely integrated physical elements: the plant frame, which supports and positions the plant-facing components; the electronic control unit, centered around the ESP32-S3 and enclosed in the 3D-printed

BMO case; and the water and lighting hardware, which connects the controller to the physical environment through the relay-controlled pump and grow light. Together, these elements provide the physical foundation for the cyber-physical feedback loop described in Section 3.

5 Software and Control Architecture

The software architecture of the Autonomous Plant Keeper is divided into three cooperating software layers: the embedded firmware running on the ESP32-S3, the cloud-connected Firebase backend responsible for higher-level processing and automatic plant-health assessment, and the React-based frontend providing the user interface.

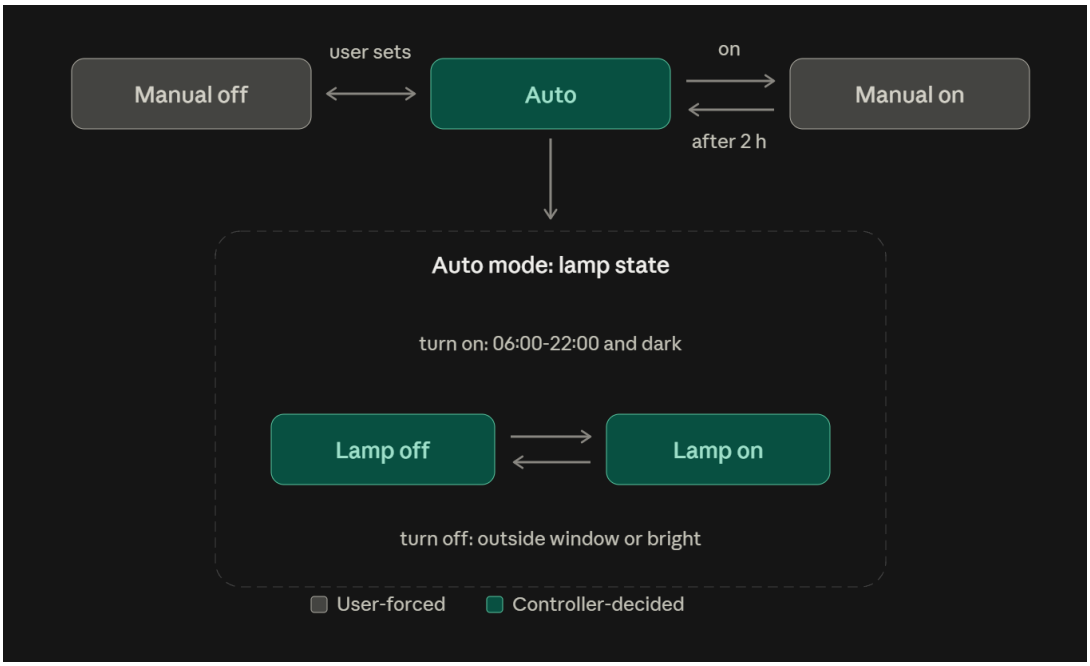
5.1 Embedded Firmware

The ESP32-S3 firmware is implemented in C++ and follows a cyclic execution model. After initialization and time synchronization, the main loop repeatedly handles communication, control requests, sensor updates, and local system-state updates. The firmware is therefore implemented as a reactive loop in which the controller periodically observes the current system state and executes the corresponding control logic.

Sensor data is sampled at regular intervals through the respective hardware components (see Section 4.1) and organized into a `SensorData` structure before being transmitted to Firebase. The firmware also periodically retrieves relevant state information from Firebase, including the requested lighting mode, pump triggers, and the latest plant-health classification. This allows the embedded controller to maintain a local representation of the relevant cyber state while remaining connected to the higher-level software components.

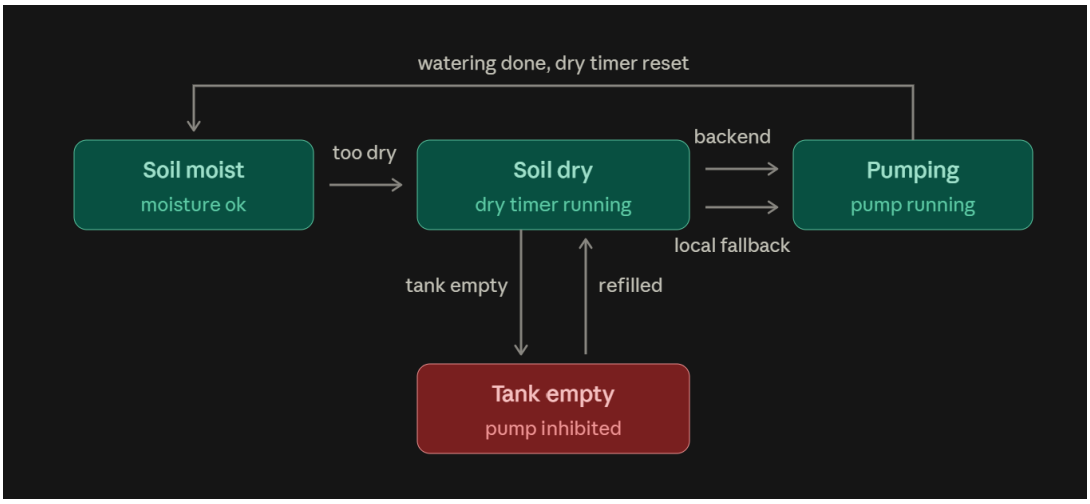
The firmware implements the main real-time control logic locally. The grow-light controller is organized around three modes, `AUTO`, `MANUAL_ON`, and `MANUAL_OFF`. In automatic mode, the controller combines synchronized local time with the measured ambient illumination to determine the lamp state. A hysteresis mechanism is used for the illumination veto, while a switching cooldown prevents excessive state changes. Temporary manual activation also expires automatically and returns the controller to automatic operation.

Grow-light Controller Modes



Watering uses a similar reactive approach. The firmware monitors soil moisture and maintains the duration for which the soil has remained below the configured threshold. Under normal operation, watering requests generated by the backend are executed when received. If the soil remains dry without a backend-triggered response for the configured grace period, the firmware can independently initiate a fallback watering cycle. The local controller therefore retains basic autonomous behavior rather than relying entirely on the cloud-based control path.

Watering Control Mechanisms



The firmware also manages the local user feedback state. It updates the display and status LED according to the current connection status and maps the received health classification to the corresponding local plant-state representation.

5.2 Firebase and Backend Integration

The communication between the embedded system and the software layer is implemented using Wi-Fi and HTTP requests to a Firebase Realtime Database. The stored record contains the environmental measurements, timestamp, water-tank state, lamp state, and effective illumination.

Control and derived state are represented by additional Firebase nodes. The Firebase `/pump` node contains a trigger and requested duration, `/light/mode` represents the desired lighting mode, `/light/state` represents the actual state reported by the ESP32, and `/health/state` contains the health classification produced by the backend. The ESP32 periodically polls these values and acts on them. After executing a pump command, it clears the trigger, preventing the same command from being executed repeatedly. Similarly, an expired manual lighting command is returned to auto by the controller.

The backend is implemented in Python using FastAPI. It communicates with Firebase through HTTP and runs independently of the frontend, including a background loop that executes approximately every 30 seconds. At each iteration, it retrieves the current sensor state, constructs the input expected by the plant-health model, performs a prediction, and publishes the resulting health classification back to Firebase.

The health classification uses a pre-trained Random Forest classifier with three possible outputs: Healthy, Moderate Stress, and High Stress. The model receives soil moisture, ambient temperature, soil temperature, humidity, and light intensity as input. In the current implementation, the measured air pressure is stored and displayed but is not used as an input feature for this model; similarly, the same measured air temperature is supplied for both the ambient-temperature and soil-temperature model features.

The same backend loop implements the primary automatic watering decision. It compares the current soil moisture value to the watering threshold and checks whether the water reservoir is empty. If the soil is sufficiently dry and the reservoir is available, the backend writes a pump trigger to Firebase. The ESP32 subsequently detects this trigger, clears it, and executes the physical pump cycle. The backend does not maintain an independent watering threshold; at startup, it reads the `WATERING_THRESHOLD` and `WATERING_PUMP_SECONDS` definitions directly from the firmware's Config.h. This establishes a single source of truth for the core watering parameters and prevents the backend and embedded controller from silently using different values.

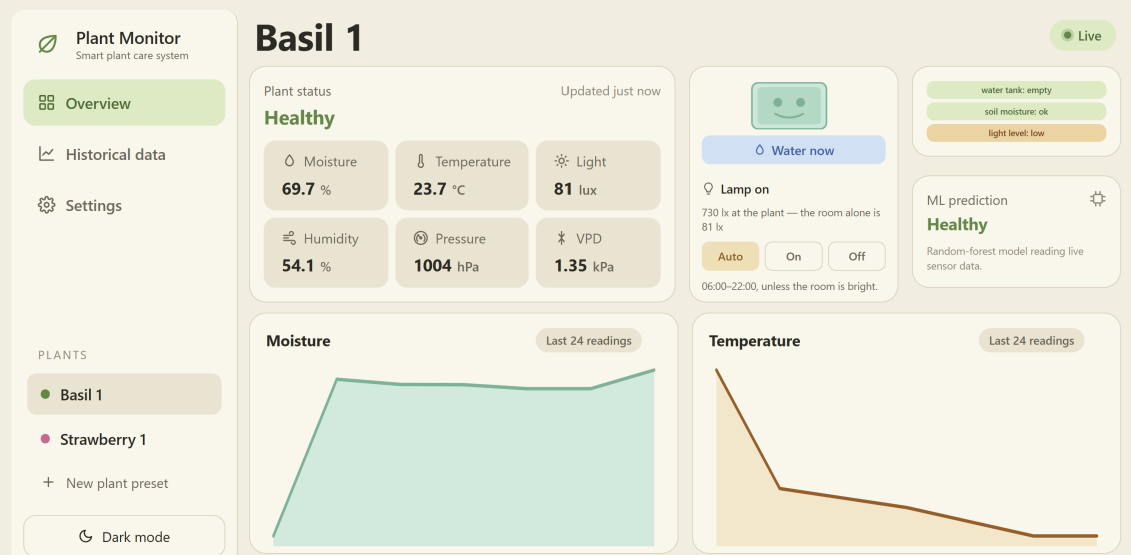
The backend also exposes REST endpoints for the frontend. `GET /plant` provides the current sensor snapshot, model prediction, pump state, and water-tank state; `GET /plant/history` provides stored historical measurements for a selected plant; and `POST /pump` creates a manual pump request.

5.3 Frontend and User Experience

The frontend is implemented using React and Vite and provides the main user-facing representation of the cyber-system. It combines direct Firebase subscriptions with periodic requests to the FastAPI backend. Firebase's realtime listeners are used for current sensor values and lighting state, allowing changes published by the ESP32 to appear directly in the dashboard. The frontend also maintains a short in-memory history of the incoming measurements, and plant-specific settings, graph selection, alert thresholds, and visual preferences are maintained locally by the frontend using browser `localStorage`. The interface supports both desktop and mobile layouts.

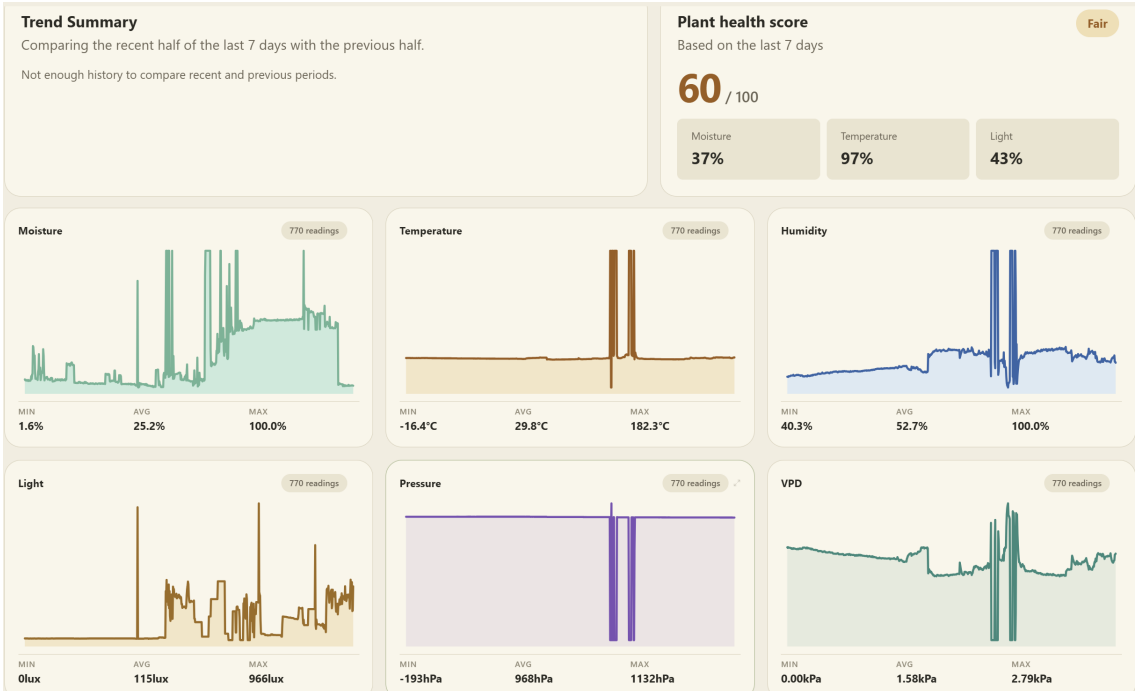
The Overview page presents the current plant status together with soil moisture, temperature, light intensity, humidity, pressure, and calculated vapor pressure deficit (VPD). The interface also displays the model's plant-health classification and provides a BMO-style visual representation whose expression corresponds to the model's health state. Notifications are generated locally by comparing selected measurements against configurable alert thresholds.

Overview page of the frontend



The Historical Data page is presented through selectable time ranges from one hour to seven days. The frontend normalizes the stored records, calculates VPD from temperature and humidity, and generates graphs for moisture, temperature, humidity, light, pressure, and VPD. It also computes a separate plant-health score from historical measurements. This score is based on the proportion of measurements satisfying the configured moisture, temperature, and light thresholds, weighted at 40%, 35%, and 25%, respectively. Trend summaries compare the recent and preceding halves of the selected time period and classify the direction of changes as rising, falling, or stable.

Historical Data Page of the frontend



The frontend also provides direct user control. A **Water now** action sends a request to the FastAPI backend, which creates the corresponding Firebase pump trigger. Grow-light control is implemented through the Firebase `/light/mode` value: selecting Auto, On, or Off changes the desired mode, which is then interpreted by the ESP32's local light controller. The interface reports the actual lamp state separately from the requested mode, allowing the user to distinguish between what was requested and what the physical controller is currently doing.

Overall, the software architecture separates responsibilities according to the nature of each task. The ESP32-S3 firmware handles direct interaction with the physical system and local control, the Firebase layer provides shared state and historical data, the back-end performs higher-level analysis and automatic watering decisions, and the frontend provides visualization and human interaction. The resulting control architecture is therefore distributed rather than centralized, allowing the physical control loop to retain local autonomy while still exposing the system's state and higher-level functionality to the cloud and the user.

6 Testing and Evaluation

The testing focuses on the core functions and interactions of the Autonomous Plant Keeper. Since the system is relatively static, the evaluation emphasizes observable changes in sensor values, actuator behavior, system responses, and the corresponding user experience. The main interactions are demonstrated in the accompanying video demo,

while this section summarizes the features tested in the demo (timestamps included below for easy reference).

T1 – Environmental Sensing (timestamp on demo: 0:40)

Description: Observe the sensor readings under normal conditions. Verify that the corresponding values on the display react accordingly.

T2 – Automatic Grow-Light Control (timestamp on demo: 1:17)

Description: Reduce the ambient illumination around the plant, and verify that the grow light is activated according to the automatic control logic.

T3 – Manual Grow-Light Control (timestamp on demo: 1:33)

Description: Use the web interface to switch the grow light between its available manual modes and observe the physical light state.

T4 – Automatic Watering (timestamp on demo: 0:45)

Description: Prepare the system with sufficiently dry soil and observe automatic watering from the pump.

T5 – Manual Watering (timestamp on demo: 1:04)

Description: Trigger watering using the Water Now function in the web interface and observe the pump activation.

T6 – Water-Reservoir Safety (timestamp on demo: 1:44)

Description: Empty or disconnect the water reservoir so that the water-level sensor indicates an empty reservoir, then attempt to activate watering. Verify that the pump does not operate.

T7 – Monitoring and Historical Data (timestamp on demo: 2:08)

Description: Operate the system and inspect the dashboard's live measurements, historical graphs, trends, and plant-health information.

7 Limitations

Although the implemented system fulfills the main functional requirements, several limitations remain. The control logic primarily addresses the expected operating conditions and only a limited number of edge cases. For example, the watering mechanism includes both a reservoir-empty check and an on-board fallback, but the system does not comprehensively handle all possible abnormal sensor readings, actuator failures, or physically unexpected situations.

A further limitation is the system's relatively limited use of edge computing. Most higher-level processing is performed by the FastAPI backend, including the Random Forest health classification and the primary automatic watering decision. The ESP32 does retain important local functionality, particularly grow-light control, reservoir safety checks, and the fallback watering mechanism. However, if the network or cloud infrastructure becomes temporarily unavailable, the embedded system cannot access the latest cloud-derived plant-health state, the most recent Firebase data, or perform the normal backend-based watering decision.

Finally, the current plant-health classification and dashboard health score should be regarded as indicators rather than definitive, detailed measurements of plant health. Despite being sufficient for casual, daily plant monitoring, neither directly observes physiological characteristics of the plant (eg. color of leaves, potential diseases), which makes the system unsuitable for users who want more fine-grained observation and control.

These limitations leave room for certain improvements in edge-side processing, local data buffering, broader fault handling, and improved plant-health modelling. However, for most casual users and plants, the system remains robust and comprehensive enough.

8 Conclusion

The Autonomous Plant Keeper fulfills the main project objectives. The ESP32-S3 continuously observes the plant environment and directly controls the light and watering hardware, while Firebase connects the embedded system with the backend and frontend. The backend provides automated plant-health classification and supports autonomous watering, while the frontend gives the user access to live and historical measurements and manual controls. The additional local control and fallback mechanisms also provide a degree of autonomy even when higher-level components are unavailable.

Overall, the project demonstrates the central principle of a cyber-physical system: information from the physical environment is sensed and processed in the cyber domain, while computational decisions and user commands can subsequently affect the physical system. The integration of these components results in a functional prototype that not only monitors plant conditions but also closes the loop between sensing, decision-making, and physical action.